

Air Service as Coordinated Resource Markets: An Industrial-Engineering Reading of the Bonded Architecture

Alessandro Daliana

May 2026

Abstract

Air service is coordination across a set of resource markets: crew-hours and certifications, aircraft-time, fuel volume, gate slots, catering counts, maintenance hours, ground handling. Each resource provider is an off-chain real-world asset (RWA) — a pilot, an airframe, a fuel inventory, a security checkpoint — whose on-chain participation is mediated by a wallet. The wallet holds the asset’s earnings as token balances and points to the asset’s off-chain credentials through NFTs (a flight-crew certification, an airworthiness certificate, an operating permit). A human or autonomous agent operates the wallet on the asset’s behalf. The wallet is an active counterparty in the assembly iff the asset is alive and its credentials are current. Public-authority services participate on the same footing as commercial providers. Bonded commitments in the Figaro kernel coordinate these wallets directly: the buyer wallet (the passenger) posts $2P_i$ at each commit, and each provider wallet posts $2G$ against the cumulative process value at the moment it commits.

Two architectural properties carry the contribution. *Asymmetric bonding within a single process*: the passenger is the **rootBuyer** of every order in the process; each provider wallet bonds against the cumulative-value accumulator G at its own commit; the kernel enforces a single **rootBuyer** across every order. *Buyer-initiated atomic resolution*: the passenger settles the entire process at destination as a single action; every active order in the process resolves together or none does.

Two consequences follow. First, the resource markets are directly accessible to the passenger through bonded commerce. Each provider wallet’s interests align with the passenger’s at the bonded commit, mediated by no third party. Second, when operational disruption occurs the seller cohort faces direct economic incentive — under locked bonds across every edge — to determine and pay compensation directly to the buyer, before any external mechanism (specialized contracts, dispute forums, insurance, regulators) engages. Cascading-delay reduction follows as a consequence of bond posture and atomic resolution; it is a property the architecture produces, not the property the paper is for.

Keywords: scheduled-service coordination, supply-chain coordination, process modeling, resource markets, weakest-link coordination, industrial engineering

1 Introduction

Air service is coordination across a set of resource markets. The journey a passenger purchases is the joint output of a flight crew (whose certifications and duty-time bands are verified at agreement signing), an aircraft (whose airworthiness certificate is verified the same way), a quantity of jet fuel, slot allocations at origin and destination gates, per-passenger catering, a share of the day’s maintenance allocation for the operating airframe, and ground handling for boarding and baggage. Each is a distinct resource the journey draws on; each has natural units of measure already in industry use — crew-hours, type ratings, fuel volume in gallons or kilograms, gate-slot time bands, meal counts, maintenance hours, baggage counts. Public services are further inputs the flight consumes: air-traffic control, security screening, the federal

aviation system itself. These public-authority services are real-world assets with commercial value in the same sense the catering supply or the fuel inventory is, and their wallets participate in the bonded process on the same footing.

Each provider in the assembly is an RWA-as-wallet. The *asset* is the off-chain real-world asset whose participation produces value — the airframe that flies, the labor capacity of the pilot, the screening service the security authority provides. The *wallet* is the on-chain representation of that participation: an address holding the asset's earnings as token balances, plus NFTs that point to the off-chain credentials the asset's role requires (the pilot's flight-crew certification, the aircraft's airworthiness certificate, the maintenance provider's authority-issued authorization). The *operator* is the human or autonomous agent who controls the wallet's signing key on the asset's behalf. The wallet is an active counterparty in the assembly iff the asset is alive and its credentials are current; that conditional is enforced operationally by the operator and at agreement signing by whichever credential schemas the assembly designer composes.

Each provider wallet sustains its participation the way a node on a blockchain network sustains its participation. A node earns fees and rewards from the network and uses them to cover its off-chain operating expenses (machines, electricity, HVAC, rent); when the receipts no longer cover the costs, the operator stops running the node and the node drops out of the network. An RWA-as-wallet operates on the same logic in its market: receipts from bonded processes must cover the asset's off-chain operating expenses (fuel, parts, labor, premises, regulatory fees, capital amortization), or the operator stops bonding the wallet into processes and the wallet drops out of the market it serves. This holds for the catering wallet and for the security-screening wallet alike: the kernel does not distinguish between a "market-traded fee" and a "regulated fee." Both are payments to service-providing wallets that need receipts sufficient to sustain operation. The schemas the assembly uses to verify credentials at agreement signing are generic across vehicle types — the same credential and schedule schemas serve aircraft, ships, trucks, and other vehicles, with per-vehicle personalization in the schema content and validator-contract configuration.

The bonded settlement primitive coordinates across these wallets without a coordinating party. Enforcement of bilateral commitments is self-enforcing through asymmetric bonding; multi-party coordination is resolvable from a single signature through atomic resolution. The journey is composed as a bonded process across those resource markets; each provider wallet bonds against the passenger's process; the passenger settles the entire process at destination through one atomic `resolveProcess` call.

Paper organization. Section 2 summarizes the settlement primitive. Section 3 develops the architectural properties applied to air-service coordination: per-edge asymmetric bonding, buyer-initiated atomic resolution, the two consequences for the assembly, the kernel / composability-stack distinction, and where specialized mechanism contracts can be developed for specific resource markets. Section 4 treats disruption resolution: the seller-cohort compensation mechanism that arises from buyer-dominance under bond pressure (the primary mechanism), illustrated through a worked bond-posture example and through the crew-allocation case, with the fallback mechanisms (specialized contracts, coordination tokens, off-chain dispute forums, insurance, regulators) treated downstream. Section 5 treats the schema requirements: a sister-schema pair for schedule binding, with the rest of the assembly satisfied by existing infrastructure that is generic across vehicle types. Section 6 treats what the architecture preserves (safety regulation, crew-labor protections, insurance, competition law) and what it changes (the cost-flow path). Section 7 states the scope conditions. Section 8 concludes.

2 The Settlement Primitive

The bonded primitive in `FigaroCore` has two mechanisms. We summarize the features the present argument uses.

Asymmetric bonding (Mechanism 1). For a transaction with payment $P > 0$ and cumulative upstream value $G \geq P$, the buyer locks $2P$ and the seller locks $2G$. Cooperation is the unique profile surviving iterated elimination of weakly dominated strategies. The mechanism scales to N -party processes through progressive collateralization: each seller bonds against the cumulative value of all upstream commitments, producing a mesh of independently secured bilateral edges, each of which carries its own equilibrium.

Buyer dominance with atomic resolution (Mechanism 2). Only the root buyer can trigger resolution, and resolution settles all active orders in the process simultaneously or not at all. The all-or-nothing rule induces a weakest-link subgame among sellers: each seller's payout depends on universal cooperation across the process. The architecture produces cooperation pressure of magnitude $P_i + 2G_i$ on every other seller from each seller's perspective, without explicit communication, governance, or managerial layer.

What this implies for air-service coordination. The bonded primitive's two mechanisms supply two coordinative properties for air service. Mechanism 1 supplies a bilateral commitment whose settlement does not require either party's home jurisdiction's contract-law regime to enforce: settlement is the on-chain release of locked bonds, mediated by no court. The off-chain agreement remains binding under whatever law the parties select for the substantive commitment, but the kernel's settlement guarantee is jurisdiction-agnostic. Mechanism 2 supplies the atomic resolution that propagates a single commit-or-don't decision through the entire process; this is what makes operational damages flow proportionally to the unit that caused them rather than being absorbed at whichever contract has the loosest cap. Neither mechanism is novel to the air-service domain; both are general-purpose settlement properties whose application here is the present paper's contribution.

3 The Bonded Architecture Applied to Air Service

The substantive contribution of the bonded primitive to air-service coordination operates at the edge of the bonded process, not at any particular position within it. Two architectural properties supply the contribution.

Asymmetric bonding within a single process. The kernel enforces a strong structural rule: every order in a process has the same buyer (the `rootBuyer`). The passenger wallet commits directly to each provider wallet. The first commit creates the process under the passenger's address; every subsequent commit in the same process extends the cumulative-value accumulator G by the new payment. At each commit the passenger posts $2P_i$ for that specific payment, and the seller posts $2G_{\text{at commit time}}$ — the full cumulative process value at the moment they commit. Bond posture is asymmetric: the passenger's per-commit bond scales with the immediate payment, while each seller's bond scales with the cumulative value of every upstream commit in the process. Sellers committing later post correspondingly larger bonds; the last seller bonds against the entire ticket value.

Buyer-initiated atomic resolution. The passenger calls `resolveProcess`, and the entire process settles simultaneously. Every active order in the process is resolved together or none is.

The passenger’s signature is the only signature required for settlement, and atomicity guarantees that no subset of orders settles without the rest.

Two consequences for the air-service assembly. The two properties together produce two consequences.

First, *the resource markets named in Section 1 become directly accessible to the passenger through bonded commerce*. Each provider wallet — the crew member wallet, the aircraft wallet, the fuel-supplier wallet, the gate-operator wallet, the caterer wallet, the maintenance-provider wallet, the ground-handling wallet — is a seller bonded against the passenger’s process, with payout depending on the passenger’s atomic resolution. Each provider’s interests align with the passenger’s at the bonded commit, mediated by no third party. We treat one alignment case in detail in Section 4: the crew-allocation arrangement.

Second, *damage flow tracks bond posture across the assembly’s commits rather than the loosest contractual liability cap*. Each seller carries asymmetric bond exposure proportional to the cumulative process value at their commit; the seller-side total grows super-linearly with the number of commits while the passenger’s posture stays at $2P_{\text{ticket}}$. The atomic-resolution rule operates on this set of bonds. When something goes wrong, every seller’s bond is locked across every commit — which is the structural pressure behind the disruption-resolution mechanism we develop in Section 4.

What the kernel constrains; what the composability stack carries. The kernel reads only per-order facts: buyer (which must be the `rootBuyer` in every order in the process), seller, payment, cumulative-value snapshot at commit, and the atomic-resolution rule covering the process. The kernel-level shape of a single process is therefore tightly constrained: one buyer, one or more sellers, all under one cumulative accumulator, all resolving atomically. Deeper compositions across multiple processes (a seller in process *A* becoming the `rootBuyer` of process *B* to coordinate their own sub-suppliers) are admissible at the kernel layer but resolve process-by-process, not across.

Above the kernel, the protocol-and-runtime composability stack attaches schemas to specific orders (the `scheduled-departure-v1` clause binds to whichever order the parties want to subject to schedule-tolerance enforcement; `handoff-v1` attestations attach to operational-handoff orders), mechanism contracts operate on specific order patterns, assembly specifications declare which schemas attach where and which view definitions render which surfaces, and UIs render the process per the assembly’s declarations. An assembly’s structure, once specified, is a fixed verifiable artifact bound to its full composability stack. The bonded-architecture argument of this paper applies to whichever resource markets the designer composes under one process, with the kernel-enforced structural rule holding throughout.

Specialized mechanism contracts per resource market. Each resource market admits its own market mechanism. The existing `DutchAuction` contract supplies one such mechanism for cases where descending-price coordination is appropriate (fuel procurement is a candidate where the supplier market admits competition). Bilateral commerce-v1 commits cover the cases where the parties contract directly. Additional mechanism contracts can be developed under the protocol-extension discipline for resources whose market mechanics deserve their own primitive: a slot-allocation mechanism for gate-time bidding, a crew-dispatch mechanism that respects type-rating and duty-time constraints, a fuel-pool mechanism for shared ramp-side procurement. The mechanism-contract space sits at the runtime tier; the kernel constraint that every order in a process shares one `rootBuyer` (the passenger) holds across whichever mechanism contracts the assembly composes.

4 Disruption Resolution Under Buyer Dominance

We turn to the operational question: when something goes wrong — a delay, a slip, a service failure — how is the disruption resolved under the bonded architecture? The substantive answer is that Mechanism 2 of the kernel (buyer dominance with atomic resolution) creates structural pressure for the seller cohort to determine and pay compensation themselves, directly to the buyer, before any external mechanism engages. This is the primary damage-management mechanism in the architecture. We treat it first, place a bond-posture worked example, develop the crew-allocation case as a concrete illustration, note the IROps-scale extension, and then position the fallback mechanisms in their proper register downstream.

4.1 The primary mechanism: seller-cohort compensation under bond pressure

Bond posture per commit. Every order in a process shares the same `rootBuyer`; the passenger as that `rootBuyer` posts $2P_i$ at each commit to each resource provider, and each resource provider posts $2G_{\text{at commit time}}$ where G is the cumulative process value at the moment of that commit. Sellers committing later in the sequence post correspondingly larger bonds. Aggregated across an N -seller process, the bonded exposure of the seller cohort scales super-linearly with the number of resource markets the assembly draws on.

A bond-posture worked example. Consider a stylized domestic short-haul passenger journey totalling \$250. The kernel enforces `buyer = rootBuyer` in every order in a process, so the passenger commits directly to each provider wallet. The ten commits below are all part of one process under one `rootBuyer` (the passenger). The kernel maintains a single process-wide cumulative-value accumulator G that grows monotonically as each commit lands ($G \leftarrow G + P_i$). At every commit the buyer posts $2P_i$ and the seller posts $2G_{\text{at commit time}}$. The asymmetry shows up across the sequence: the passenger’s buyer-side bond at each commit scales only with the immediate payment to that seller, while each seller’s bond scales with the full cumulative value of every upstream commit in the process. Sellers committing later post correspondingly larger bonds; the last seller bonds against the entire ticket value. This is progressive collateralization in operation. Table 1 traces the sequence.

Two features of the table carry the structural argument.

The asymmetry between buyer and seller bonds. The passenger’s aggregate locked bond is \$500, which equals $2 \times P_{\text{ticket}}$ — the standard buyer posture. The seller cohort’s aggregate locked bond is \$3,512, more than seven times the passenger’s posture, because each seller bonds against the cumulative G that has grown across the upstream commits. This is not an accident of the example; it is the asymmetric-bonding architecture in operation. As the assembly adds more resource markets, the seller-side total grows super-linearly with the number of commits while the passenger’s side stays at $2P_{\text{ticket}}$.

The magnitude of the seller cohort’s locked bond. The seller cohort’s aggregate locked bond (\$3,512) is more than fourteen times the ticket payment. A non-resolving passenger holds that full pool locked across every commit in the process. The pressure on the cohort to compensate the passenger and secure resolution is structural in proportion that exceeds the ticket by more than an order of magnitude.

The order in which sub-orders commit reorders which specific seller posts the largest bond, but does not change the aggregate seller cohort total: it depends only on the set of payments, not the sequence. Whichever seller commits last bonds against the full $G_{\text{final}} = P_{\text{ticket}}$.

Withholding resolution as the buyer’s leverage. The passenger holds unilateral authority to call `resolveProcess`. As long as the passenger does not call, every seller’s bond in the process remains locked. Each seller faces forfeit of their full $2G_i$ on continued non-resolution. The buyer’s

Seller (commit sequence)	P_i	G after commit	Passenger bond $2P_i$	Seller bond $2G$
Aircraft wallet (root)	\$65	\$65	\$130	\$130
Fuel-supplier wallet	\$45	\$110	\$90	\$220
Crew member wallet	\$40	\$150	\$80	\$300
Maintenance-provider wallet	\$15	\$165	\$30	\$330
Catering wallet	\$5	\$170	\$10	\$340
Ground-handling wallet	\$15	\$185	\$30	\$370
Origin airport authority wallet	\$20	\$205	\$40	\$410
Destination airport authority wallet	\$20	\$225	\$40	\$450
TSA wallet (security screening)	\$6	\$231	\$12	\$462
Federal aviation system wallet	\$19	\$250	\$38	\$500
Total bond locked across the process	—	—	\$500	\$3,512

Table 1: Stylized per-commit bond posture for a domestic short-haul passenger journey under one process. The passenger commits directly to each provider wallet (the kernel enforces buyer = `rootBuyer` in every order). G is the process-wide cumulative-value accumulator, monotonically growing with each commit. The passenger’s bond at each commit is proportional to the immediate payment; each seller’s bond is proportional to the cumulative process value at the moment that seller commits. The last seller to commit therefore posts the largest seller bond, $2G_{\text{final}} = 2 \times \$250 = \$500$. Per-seller payments are illustrative; absolute magnitudes vary with route, aircraft type, and operating context.

atomic-resolution authority is therefore not a remedy in the conventional sense (it does not pay the buyer anything beyond bond recovery); it is leverage. The buyer can hold the entire bond pool locked by simply not signing.

Sellers determine and pay compensation themselves. When operational disruption occurs and the buyer is left worse off than the agreement contemplated, the seller cohort faces direct economic incentive to make the buyer whole enough that the buyer chooses to resolve. The mechanism unfolds operationally as follows.

Sellers approach the buyer with compensation offers. Compensation takes whatever form the buyer accepts: cash transfer, future service credit, accommodation or meal vouchers, refund of the ticket payment, alternative routing, partial or full refund combined with services. The currency of compensation is at the parties’ discretion — the kernel does not constrain it.

Sellers negotiate among themselves about which seller bears what fraction of the compensation. The unit whose performance materially failed bears more. Operational parties around the failure who could not have prevented it (the catering provider who had no role in a fuel slip; the gate operator whose service was unaffected) contribute less or not at all. The negotiation is real but routine: each seller knows their own bond exposure, their counterfactual payout under cooperative resolution, and their willingness-to-pay against losing the bond entirely.

The buyer accepts the compensation that satisfies them and calls `resolveProcess`. Bonds release across the process; every seller recovers $2G_i + P_i$ less their share of the compensation; the buyer recovers their bond plus the compensation already paid. The settlement is bilateral and direct.

This happens fast. The negotiation occurs at the operational boundary (the airport gate, the destination airport’s customer service desk, in some cases through the runtime UI’s disruption-coordination surface), not in a court filing six months later. It happens before insurance claims, before dispute-resolution forums, before regulatory engagement. Most disruption resolves through this path because that is where the bond pressure points.

The crew-allocation case. The mechanism is concrete in the crew market. The crew-allocation operates as a direct market between the passenger and the crew member. The crew member’s wallet — holding their certification NFTs and operating-permit credentials, verified through the relevant credential schemas at agreement signing — bonds against the passenger’s process at the crew-edge. The crew member’s payout depends on the passenger’s atomic resolution at destination; the crew member’s interests align with the passenger’s directly through the bonded edge. The allocation discipline is whichever mechanism contract the assembly designer composes for the crew market: a bilateral commerce-v1 commit, a descending-price **DutchAuction**, or a future crew-dispatch mechanism contract that respects type-rating and duty-time constraints. Collective-bargaining institutions over wage rates, duty-time protections, and seniority among crew remain; they are labor-market institutions the bonded primitive does not touch.

Under disruption — say a crew member cannot fly — the crew-edge bond is locked alongside every other edge in the process. The crew member negotiates compensation directly with the passenger (find a substitute crew member at speed, accommodate the passenger on the next flight, refund the leg, or whatever the passenger accepts). Resolution follows compensation. There is no intermediary in this negotiation; the architecture does not contemplate one.

IROps-scale coordination. The single-flight example understates the operational complexity of large-scale irregular operations (IROps). A hub disruption produces simultaneous bonded processes across hundreds of flights, with thousands of bonded edges across the resource-market mesh. The compensation negotiation at IROps scale is operationally heavier than the per-flight case: each affected passenger holds a separate bonded process with its own seller cohort, and many provider wallets (crew member wallets, aircraft wallets, ground-handling wallets) appear in many processes at once. The bonded architecture does not eliminate this coordination cost; it routes the cost in a structurally different direction. Each affected passenger faces a busy but bilaterally coordinated seller cohort negotiating their compensation directly. Operational tooling for the provider wallet’s operator — who manages bonds locked across many simultaneous processes and must coordinate compensation responses at scale — is itself an assembly-design question.

The weakest-link character of the pressure. Under buyer dominance with atomic resolution, each seller bears direct economic interest of magnitude $P_i + 2G_i$ in every other seller’s performance, because each seller’s payout depends on universal cooperation across the process: cooperation is the unique equilibrium surviving iterated elimination of weakly dominated strategies, induced by atomic resolution operating on the bonded mesh. The analytical content is the formal characterization of the pressure; the operational content is sellers paying compensation to buyers directly to secure resolution. The same structural shape produces the joint-liability dynamic in Grameen-style microfinance, in which group members chip in to cover a struggling member’s installment before the lender forecloses, because each member’s continued credit access is exposed to every other member’s repayment. The bonded architecture’s seller-cohort-compensates-buyer dynamic is the same structural mechanism scaled with chain depth and operating without the social-substrate prerequisites Grameen requires. Buyer dominance with atomic resolution is in this sense a *social mechanism*: it produces social-coordination behavior (peer pressure, cohort negotiation, burden-sharing, coverage of a struggling counterpart) endogenously through the bond architecture, with each provider wallet’s continued participation in its market making the negotiation rationally compelled rather than socially expected.

Why this is structural, not procedural. The mechanism is not a courtesy that sellers may or may not extend. It is a direct consequence of the bond posture under atomic resolution. A seller who refuses to participate in compensation negotiation watches their full bond forfeit

alongside every other seller’s. The compensation mechanism emerges from the architecture because the architecture has made non-cooperation strictly worse than reasonable compensation for every seller in the process.

4.2 Fallback mechanisms

The primary mechanism resolves most disruption directly. Fallback mechanisms engage when the primary does not, or when the disruption involves a class of issue the primary cannot address. We list them in the order of operational engagement.

Specialized mechanism contracts (protocol layer). Where the assembly composes a specialized mechanism contract — a delay-compensation contract that disburses pre-agreed amounts under specified conditions, a slot-reallocation mechanism, a fuel-pool mechanism — the contract handles its specific coordination automatically. The specialized contract complements the primary mechanism by making specific compensation patterns automatic rather than negotiated.

Coordination tokens (runtime layer). Where the assembly composes around a coordination token, the substrate’s operational state registers as token value. Token holders bear operational quality through value impact rather than through direct compensation. Token-value reflection is a diffuse signal operating in parallel with the per-edge compensation mechanism, not a substitute for it.

Off-chain dispute resolution forums. The off-chain agreement named in the bonded commitment binds a dispute forum (state court, international arbitration center, ODR platform, Schelling-point juror system). The forum engages when the seller-cohort compensation does not resolve the dispute — typically because the dispute is substantive in a way bond pressure cannot address (duress, mistake, frustration, illegality, public-policy concerns). The bonded record is the evidentiary input the forum operates on: kernel byproducts (commit and resolution events) supply the indisputable settlement record, and schema-validated attestations (**handoff-v1** timestamps, **delivery-lifecycle-v1** stage events) supply the substantive performance evidence.

Insurance compositions. A passenger or provider wallet who wants insurance against risks the bond pool cannot absorb (catastrophic events, multi-day weather disruption, force-majeure events) composes a parallel bonded process with an insurer. The insurer’s bond underwrites the risk; insurance settlement is its own bonded event. Insurance composes with the bonded primitive without modifying the kernel.

Regulators. Aviation safety regulators, consumer protection authorities, antitrust enforcement, and labor protection regimes operate alongside the bonded architecture and engage in the matters their authority covers. The architecture does not displace them.

The substantive contribution of the bonded architecture to damage management is that the primary mechanism — seller cohort negotiating and paying compensation directly to the buyer, before external mechanisms engage — is structural, immediate, and operates without any of the fallbacks being needed in most cases. The conventional regime’s cascading-delay pathology — damages absorbed by whichever contract has the loosest liability cap, with limited unit-level operational signal [Lan et al., 2006] — reduces under the bonded architecture as a consequence of bond posture and atomic resolution.

5 Schemas

The assembly draws almost entirely on existing schema infrastructure. A sister-schema pair is required for schedule binding: **scheduled-departure-v1** and **scheduled-arrival-v1**, so the resolution path can verify whether the sellers delivered against the agreed departure and arrival schedule respectively.

The schedule and credential schemas the assembly relies on are generic across vehicle types: the same primitives serve aircraft, ships, trucks, and other scheduled vehicles, with per-vehicle

personalization in the schema content (designator format, authority issuing the credential, tolerance windows) and in validator-contract configuration. The air-service application below is one personalization of those primitives, not a domain-specific architecture.

Existing schemas. `figaro-commerce-v1` carries the currency, payment, and itemized line-items of every commitment in the assembly — the passenger ticket and every sub-procurement commitment. `figaro-delivery-lifecycle-v1` carries staged progression as a sequence of attestations; the schema’s five-slot structure can be re-used by reinterpretation (boarding initiated, cabin ready, pushback, takeoff, arrival, or whatever projection the assembly designer adopts), without authoring a new schema. `figaro-handoff-v1` carries the operational handoffs between operational parties. `figaro-jurisdiction-v1` carries the off-chain forum-selection clause for international flights, where the parties want to specify a particular dispute-resolution forum. `figaro-geo-v1` carries the origin and destination geohashes where geographic-anchoring is operationally relevant. The GHG family of schemas is available for assemblies that wish to compose carbon-emission attestations into the ticket commitment.

The credential schemas verifying the wallet-held NFTs that qualify each provider to participate (the pilot wallet’s FAA certification, the aircraft wallet’s airworthiness certificate, the maintenance-provider wallet’s authority-issued certifications) are runtime-tier credential schemas the assembly designer composes; the kernel does not enforce them, but the validator contracts the assembly binds at agreement signing do.

The new sister-schema pair. The pair binds a `commerce-v1` ticket commitment to a specific scheduled departure event and a specific scheduled arrival event, so the resolution path can verify whether the sellers delivered against schedule at each end of the journey. Splitting departure and arrival into a sister pair follows the established sister-schema pattern in Figaro (proximity-policy / proximity-proof; the GHG-disclosure family / GHG-measurement split): each event is attested separately, with content shaped to the event the parties expect at that point.

`scheduled-departure-v1` (Layer A spec):

- `flightDesignator` (string, IATA format, e.g. “NK1234”)
- `scheduledDeparture` (ISO 8601 datetime, with explicit timezone)
- `originAirport` (string, ICAO 4-character, e.g. “KORD”)
- `aircraftType` (string, optional, IATA aircraft code; omitted if the parties reserve substitution rights at the runtime tier)
- `departureToleranceSeconds` (integer, the agreed slip below which no penalty applies; e.g. 900 for a 15-minute window)

`scheduled-arrival-v1` (Layer A spec):

- `flightDesignator` (string, IATA format)
- `scheduledArrival` (ISO 8601 datetime, with explicit timezone)
- `destinationAirport` (string, ICAO 4-character)
- `aircraftType` (string, optional, IATA aircraft code)
- `arrivalToleranceSeconds` (integer)

The two schemas share the `flightDesignator` field so the arrival event can be matched to its departure within the bonded process. We deliberately do not impose a maximum on `departureToleranceSeconds` or `arrivalToleranceSeconds`. Scheduled-service operations vary widely in operationally meaningful tolerance windows, and hard-coding a maximum at the validator contract would foreclose legitimate use cases under the first-write-wins binding discipline that makes the `schemaId` permanent. The contracting parties choose the tolerance their operation supports; if a domain-specific maximum is later wanted, it can be enforced at a tighter sister schema (e.g. `figaro-passenger-departure-v1`) without mutating the present binding. The pair is the seed of a *scheduled-service* family; later extensions could cover scheduled ground transport, scheduled deliveries with tight windows, and other scheduled-binding use cases without reauthoring the family.

Schemas the architecture deliberately does not include. Three classes of schema are conspicuously absent. First, there is no force-majeure schema that allows a counterparty to avoid bond loss in weather events. Force-majeure handling is a *composition* concern, not a kernel concern: a party who wants to insure against weather-driven delay composes a parallel insurance process; the kernel neither knows about weather nor adjudicates whether a given event qualifies. This is the no-escape-hatches discipline of the kernel, applied to the air-service domain. Second, there is no schema that governs which delays count toward bond loss; the `scheduled-departure-v1` and `scheduled-arrival-v1` tolerances are the kernel’s only such criteria, and any further refinement is for the off-chain forum that adjudicates disputes the parties bring to it. Third, there is no schema that grants any party a unilateral reschedule right; rescheduling is a new commitment with a new `schemaId` or a new instance, not a mutation of the existing binding. Append-only identity is the discipline.

6 Comparison with the Conventional Apparatus

The architectural change should be located precisely against the existing apparatus to clarify what does and does not change.

What changes. The architecture changes the cost-flow path. Under the contract-of-carriage regime, passenger schedule loss is bounded by per-incident penalty caps and counterparty discretion in issuing voucher compensation; sub-supplier liability for sub-service slip is bounded by per-incident contractual penalties; and the residual loss is absorbed by whichever contract has the loosest cap. Under the bonded architecture, schedule loss is allocated proportionally to the wallet whose bond is exposed at the relevant process layer: sub-supplier bond for sub-service slip, root-counterparty bond for arrival-tolerance violation, passenger bond returned in full on satisfactory delivery. The cost flow is direct, proportional, and tracks the operational layer at which performance failed.

What does not change. Safety regulation is unchanged: the FAA in the US, EASA in the EU, and analogous national civil-aviation authorities continue to certify aircraft, pilot licensing, maintenance procedures, and operational standards. Crew-labor protections are unchanged: pilot-union and flight-attendant collective bargaining and flight-time / duty-time limits under FAR Part 117 in the US (with analogous regimes elsewhere) apply to the crew-edge under the bonded architecture exactly as they apply under the conventional regime. Insurance and reinsurance are unchanged in their underlying function; the bonded architecture changes the risk profile aviation insurance markets price (a smaller residual after bond allocation; concentrated risk at whichever edges carry the largest cumulative-value bond posture; a new class of bond-default risk at every provider wallet in the process) but does not displace the insurance function.

Antitrust and competition law are unchanged: provider-wallet markets remain subject to anti-collusion enforcement, slot-allocation regimes at congested airports remain governed by the relevant authorities, and code-share arrangements remain subject to the applicable competition regimes.

Regulatory roles attached to wallets. Some regulatory functions attach to a single designated holder in the conventional regime — FAA Part 121 (and analogous regimes in other jurisdictions) imposes a single-certificate-holder requirement on commercial scheduled passenger operations, with many safety functions attaching to that holder. Under the bonded architecture the certificate-holder is one of the wallets in the assembly (typically the aircraft wallet, or a wallet associated with the aircraft’s operating credentials); the certificate function is preserved, but the certificate-holder participates as one bonded counterparty alongside others, not as a cross-market coordinator. Off-chain discovery surfaces, quality-signal aggregators, and multi-process curators may also be useful around the bonded process. The kernel forbids any party other than the passenger from being the buyer of resource-market commits, so no off-chain surface can re-emerge as a kernel-level coordinator.

What this is not. The architecture is a re-architecture of the cost-flow path in air-service coordination. Whether any specific provider wallet operates well or poorly within the architecture is an operational-management question the architecture is silent on.

The architecture is also not a passenger-protection regime. It is a coordination protocol that has the side effect of routing schedule-loss damages to the passenger more proportionally than the contract-of-carriage regime does. Passenger-protection regimes (the EU’s Regulation 261/2004, the US’s Department of Transportation refund and compensation rules, and analogous regimes elsewhere) continue to operate in their own register and apply to the regulated wallet in the relevant jurisdiction. The bonded architecture’s passenger-side benefit is structural rather than regulatory.

7 Scope

The argument bounds itself by several scope conditions.

Asset specificity. The architecture works best where provider wallets compete in markets with multiple available alternatives. Where a provider is a near-monopolist (as airport authority wallets often are at hub airports for gate operations, or as ATC services are by regulatory construction), the buyer-side leverage at sub-procurement is constrained. The architecture is most effective in the low-specificity region of the resource portfolio and least effective at the high-specificity end.

Bond-currency access. The architecture presumes that all parties can post collateral in a denomination the kernel accepts. For some provider wallets (e.g. small catering contractors operating in jurisdictions with limited crypto on-ramps) this may be a binding operational constraint. The constraint is real but solvable through standard treasury and on-ramp infrastructure.

Aggregate liquidity at the provider tier. A high-volume provider wallet covering thousands of flights per day under the bonded architecture would face aggregate bond-custody requirements that scale with daily volume. This is an operational treasury question: the wallet maintains a working bond pool sized to its active commitments, with bonds released and rebound as processes resolve. The cumulative custody at any moment is bounded by the active-process count multiplied by per-process bond requirements; the architecture is liquidity-intensive in steady state but not unbounded.

Composability with the conventional regime. The architecture is composable with the conventional regime: a provider wallet can serve some passengers under the bonded architecture and others under the contract-of-carriage regime, and a sub-supplier can serve some counterparties under bonded contracts and others under conventional ones.

8 Conclusion

Air service is a coordination problem across resource markets. Each resource provider is a wallet holding productive value: NFT credentials that qualify the wallet to participate, the underlying real-world asset whose lifecycle the wallet tracks, and a signing key that binds the wallet to bonded commitments. The bonded primitive in **FigaroCore** coordinates across those wallets without a third party: each commit binds a bilateral edge, asymmetric bonding makes cooperation the dominant strategy at every edge, and atomic resolution propagates a single commit-or-don't decision through the entire process. Cascading-delay reduction follows as a consequence of bond posture and atomic resolution; it is a property the architecture produces, not the property the paper is for.

The architecture is composable with existing safety regulation, crew-labor protections, insurance markets, and competition regimes. It changes the cost-flow path without displacing the regulatory and contractual layers that surround it. The architecture is well-defined, and its operational properties differ from the conventional regime in ways the operations-research literature on airline scheduling has documented without being positioned to address. What the present paper supplies is the assembly — the configuration of bonded commits, the credential schemas, the schedule sister-pair, and the disruption-resolution mechanism — that turns the bonded primitive into a working air-service coordination architecture.

Acknowledgments

The worked example originated in a project repository of agent-coordination scenarios; the present paper formalizes the scenario into an industrial-engineering register.

References

- Lan, S., Clarke, J.-P., and Barnhart, C. Planning for Robust Airline Operations: Optimizing Aircraft Routings and Flight Departure Times to Minimize Passenger Disruptions. *Transportation Science*, 40(1):15–28, 2006.
- European Parliament and Council. *Regulation (EC) No 261/2004 establishing common rules on compensation and assistance to passengers in the event of denied boarding and of cancellation or long delay of flights*. February 11, 2004.
- US Federal Aviation Administration. *14 CFR Part 117: Flight and Duty Limitations and Rest Requirements: Flightcrew Members*. 2014.